

AI-Driven QA & Test Automation Problem Statements

Overview • Problem Statement • Delivery Impact • Evaluation Parameters

Prepared from the attached QA / automation problem-statement notes

Executive Summary

The problem statements below convert the ideas in the supplied notes into structured initiatives that can be used for hackathons, innovation programs, QA transformation roadmaps or engineering proposals. Each initiative keeps human approval where a change could affect product behaviour or test integrity.

Common Evaluation Principles

- Business value: measurable reduction in delivery effort, risk or cycle time.
- Quality improvement: better coverage, fewer escaped defects and stronger release confidence.
- Automation effectiveness: percentage of repetitive QA work that can be safely automated.
- AI quality: accuracy, explainability, confidence thresholds and human-approval rate.
- Operational adoption: ease of integration with existing Jira, Confluence, Test management, CI/CD and automation frameworks.
- Governance: auditability, traceability, rollback and protection against false fixes.

1. AI-Powered Unified Self-Healing Automation with Jira/Test management Integration

1. Overview

Build an AI-powered unified self-healing automation capability across Web, API and Mobile automation. The solution should detect automation failures, identify whether they are caused by automation drift or genuine product defects, perform validated self-healing where possible, and integrate the complete workflow with test management and Jira.

2. Problem Statement with Description

Automation failures across web journeys, APIs and Mobile applications frequently occur because of application changes that do not necessarily represent a product defect. Web applications may change locators, DOM structures, attributes or page flows; APIs may change request/response structures, fields, status codes or schemas; and Mobile applications may change accessibility IDs, dynamic identifiers, element properties or UI flows.

Currently, QA engineers often need to manually inspect logs, screenshots, stack traces, API responses and application changes to determine the root cause, update automation scripts and re-execute the failed test. Test-case updates in test management and Jira defect creation are also often performed manually. This increases maintenance effort, creates regression delays and can result in false failures or inconsistent defect reporting.

The proposed solution should read the approved test case from test management, execute the corresponding automation, analyze failures using AI and classify them as automation drift, flaky/environment/data issues or potential product defects. For automation drift, it should identify alternative locators, element properties, interaction strategies or API validation/contract updates, apply the repair under defined confidence thresholds, validate the repair through re-execution and update the relevant test

management test-case execution/result. For genuine product failures, it should automatically create a Jira defect containing the failure evidence and link/reference it from the test management execution.

The complete workflow should maintain traceability from test case to automation execution, failure analysis, self-healing or defect creation, re-execution and result, with human approval for low-confidence or potentially risky changes.

3. Expected Test Automation solution from Candidate (Web, API and Mobile Automation)

Manual Test Case → Test automation Script generation (Using tools like Python, Selenium, Playwright) → Integration with Git (Use any personal or public Git for demo) → Automation Execution → Web / API / Mobile → Failure Detection → AI Analysis & RCA → Automation Drift? → YES: Self-Healing → Re-execution → Update in test management | NO: Jira Defect Creation → test management Update → Audit Trail

4. Expected Impact on Delivery by Solving the Problem

- Reduce manual maintenance effort across Web, API and Mobile automation.
- Reduce false failures caused by locator, UI, element-property and API contract changes.
- Improve stability of critical Web regression journeys.
- Reduce automation failure investigation and recovery time.
- Automatically maintain traceability between Test management test cases, automation execution and Jira defects.
- Reduce manual effort involved in creating and updating Jira defects.
- Improve accuracy and consistency of failure classification and root-cause analysis.
- Automatically validate self-healing changes through controlled test re-execution.
- Improve regression-suite availability and CI/CD pipeline stability.
- Provide faster feedback to QA, development and product teams.
- Create an auditable end-to-end QA workflow from test execution through defect reporting.

5. Evaluation Parameters

- Overall self-healing success rate
- Web self-healing success rate
- API self-healing / patch acceptance rate
- Mobile self-healing success rate
- Failure classification accuracy
- False-failure / flaky-test reduction
- Manual automation-maintenance hours saved
- Average recovery time per failed test
- Test re-execution success rate after self-healing
- Test management results updating
- Jira defect creation accuracy
- Duplicate Jira defect reduction
- AI recommendation / repair acceptance rate
- Reduction in repeated automation failures
- Execution reports dashboard